

# FHIR Integration

## 1. Overview

### 1.1 Description

**FHIR Integration** connects any **HL7 FHIR R4**-compliant scheduling/EHR system (reference implementation: **Aidbox**) with the **Newo Platform**.

It enables:

- Checking available appointment slots for a pre-selected provider and facility
- Booking appointments with automatic patient lookup and creation
- Cancelling existing appointments
- Pre-configured facility ( `Organization` ) and provider ( `PractitionerRole` ) selection during setup
- OAuth2 token management with `authorization_code` or `client_credentials` grant and automatic refresh

This integration is designed for **clinics, medical practices, and any scheduling-capable EHR that exposes a FHIR R4 REST API** who need **automated appointment scheduling through a conversational AI agent** (voice or chat).

Because the integration speaks standard FHIR REST, it is **vendor-agnostic**: the same skills work against Aidbox, Epic FHIR, Cerner/Oracle Health, Azure FHIR, Google Cloud Healthcare API, HAPI FHIR, or any other conformant server — only the base URL and OAuth endpoints change.

### 1.2 Use Cases

- When a patient asks about available slots → AI queries `Schedule` for the selected provider and then `Slot` for free time windows
- When a patient wants to book an appointment → Search `Patient` by phone, create one if not found, then create an `Appointment` resource
- When a patient wants to cancel an appointment → Patch the existing `Appointment` resource with `status = cancelled`
- When a new patient books → Automatically create a `Patient` record in FHIR before the appointment is written
- When the project is published → Facilities ( `Organization` ) and providers ( `PractitionerRole` ) are fetched and exposed as enum dropdowns in the Builder

### 1.3 Supported Features

| Feature            | Supported | Notes                                                                               |
|--------------------|-----------|-------------------------------------------------------------------------------------|
| Check Availability | Yes       | Date range lookup via <code>Schedule + Slot?status=free</code>                      |
| Book Appointment   | Yes       | With automatic Patient creation if no record matches the caller                     |
| Cancel Appointment | Yes       | <code>PATCH /Appointment/{id}</code> with <code>status = cancelled</code>           |
| Facility Selection | Yes       | Pre-configured during setup via <code>Organization?active=true&amp;type=prov</code> |

|                               |     |                                                            |
|-------------------------------|-----|------------------------------------------------------------|
| Provider Selection            | Yes | Pre-configured during setup, filtered by selected facility |
| Patient Search                | Yes | GET /Patient?phone=...                                     |
| Patient Creation              | Yes | Auto-creates during BookingFlow if no match is found       |
| Token Auto-Refresh            | Yes | OAuth2 refresh_token with fallback to client_credentials   |
| OAuth Authorization Code Flow | Yes | One-time interactive login via hosted redirect page        |
| Client Credentials Grant      | Yes | Headless alternative for system-level integrations         |
| Multi-Provider Scheduling     | No  | A single provider is selected during setup                 |
| Reschedule Appointment        | No  | Cancel + rebook as workaround                              |
| Multiple Locations            | No  | One Organization is selected per project                   |
| Historical Data Import        | No  | Only events created after activation are handled           |

## 2. Requirements

Before installation:

- Access to a **FHIR R4**-compliant server that supports the following resources: `Organization` , `PractitionerRole` , `Patient` , `Schedule` , `Slot` , `Appointment`
- **OAuth2 Client ID** and **Client Secret** registered with the FHIR server
- The **Base URL** of the FHIR server (without the trailing `/fhir` segment)
- SMART-on-FHIR scopes granted on the server side:
  - `system/Patient.read` / `system/Patient.write`
  - `system/Organization.read`
  - `system/PractitionerRole.read`
  - `system/Schedule.read`
  - `system/Slot.read`
  - `system/Appointment.write`
- The FHIR server must be reachable over HTTPS from the Newo Platform

## 3. How to Setup

### 3.1 Step 1 — Register an OAuth2 Client

1. Log in to your FHIR server's admin console (Aidbox, Azure Health Data Services, Epic App Orchard, etc.)
2. Register a new OAuth2 client application
3. Add the Newo redirect URL: `https://static.newo.ai/oauth/index.html`
4. Request the SMART scopes listed in section 2
5. Note the **Client ID** and **Client Secret**

### 3.2 Step 2 — Connect in Platform

1. Open **Newo** → **Projects**
2. Set the following attributes in **Builder / Attributes**:

| Attribute                           | Required | Description                                                                                                                                                                   |
|-------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| fhir_base_url                       | Yes      | FHIR server base URL (e.g., <a href="https://fhir.example.com">https://fhir.example.com</a> ). The integration appends /fhir/... for resource calls and /auth/token for OAuth |
| fhir_client_id                      | Yes      | OAuth2 Client ID                                                                                                                                                              |
| fhir_client_secret                  | Yes      | OAuth2 Client Secret                                                                                                                                                          |
| fhir_grant_type                     | Yes      | authorization_code (interactive) or client_credentials (headless)                                                                                                             |
| fhir_oauth_code                     | Yes*     | One-time authorization code; required only when fhir_grant_type = authorization_code. Obtain via the hosted login link shown in the attribute description                     |
| fhir_scope                          | Yes      | OAuth2 scope list (see section 2)                                                                                                                                             |
| fhir_facility                       | No       | Facility (Organization) — auto-populated during setup as an enum                                                                                                              |
| fhir_provider                       | No       | Provider (PractitionerRole) — auto-populated after facility is selected                                                                                                       |
| fhir_enable_slot_check              | No       | Enable availability checks (default: True)                                                                                                                                    |
| fhir_enable_booking                 | No       | Enable appointment creation (default: True)                                                                                                                                   |
| fhir_enable_cancellation            | No       | Enable cancellations (default: True)                                                                                                                                          |
| fhir_check_existing_client          | No       | Search Patient by phone before creating a new record (default: True)                                                                                                          |
| fhir_show_for_days_customer         | No       | Number of days ahead to search for available slots (default: 5)                                                                                                               |
| fhir_use_timezone_from_conversation | No       | If True, detect caller time zone from conversation; if False, use business TZ                                                                                                 |
| fhir_override_agent_attributes      | No       | Override SuperAgent attribute schemas on publish (default: True)                                                                                                              |
| fhir_access_token                   | No       | Populated automatically after successful OAuth exchange                                                                                                                       |
| fhir_refresh_token                  | No       | Populated automatically after successful OAuth exchange                                                                                                                       |

3. If `fhir_grant_type = authorization_code` :

1. Open the **FHIR OAuth Code** ( `fhir_oauth_code` ) attribute — its description contains a clickable hosted login link
2. Log in to the FHIR server, confirm the requested scopes

3. After redirect, copy the returned code and paste it into the attribute
4. Click **Save + Publish All**
5. On publish, the **SetupFlow** automatically:
  - Exchanges the credentials for an `access_token` + `refresh_token` via `POST /auth/token`
  - Creates `fhir_connector` (resource calls) and `fhir_token_connector` (token calls)
  - Fetches all facilities with `GET /fhir/Organization?active=true&type=prov` and populates the facility selector
  - Fetches providers for the selected facility with `GET /fhir/PractitionerRole?organization:identifier={facility_id}` and populates the provider selector
  - Registers availability, booking, and cancellation tools with the ConvoAgent
  - Injects payload schemas for the ConvoAgent tool-calling framework

## 4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

### 4.1 How the Integration Works

The integration uses the standard FHIR REST API for scheduling and demographic data. Unlike multi-provider integrations, the facility and provider are pre-selected during setup, which keeps runtime queries simple and deterministic. The access token is automatically refreshed on `401` responses using the `refresh_token` grant, with a fallback to `client_credentials` when no refresh token is available.

- A **Trigger** is a system event that starts a flow
- An **Action** is the FHIR REST operation executed against the server

#### Available Triggers

| Trigger                   | Event                                   | Description                                           |
|---------------------------|-----------------------------------------|-------------------------------------------------------|
| <b>Check Availability</b> | <code>get_available_slots</code>        | When the agent needs to look up free slots            |
| <b>Book Appointment</b>   | <code>book_slot</code>                  | When the agent confirms a booking                     |
| <b>Cancel Appointment</b> | <code>convo_agent_cancel_booking</code> | When the agent processes a cancellation               |
| <b>Setup</b>              | <code>project_publish_finish</code>     | Initializes the integration on project deploy         |
| <b>Execute Request</b>    | <code>fhir_execute_request</code>       | Internal — every resource call goes through UtilsFlow |

#### Available Actions

- **Get Facilities** — `GET /fhir/Organization?active=true&type=prov`
- **Get Providers** — `GET /fhir/PractitionerRole?organization:identifier={facility_id}`
- **Get Schedule** — `GET /fhir/Schedule?date=ge{from}&date=le{to}&sactor:identifier={provider_id}`
- **Get Slots** — `GET /fhir/Slot?schedule:identifier={schedule_id}&status=free&start=ge{from}&start=le{to}`
- **Search Patient** — `GET /fhir/Patient?phone={phoneNumber}`
- **Create Patient** — `POST /fhir/Patient` with a minimal `Patient` resource (name, `telecom` phone + email)

- **Create Appointment** — POST /fhir/Appointment with status = proposed , start/end, provider and patient participants
- **Cancel Appointment** — PATCH /fhir/Appointment/{id} with JSON Patch [{ "op": "replace", "path": "/status", "value": "cancelled" }]
- **Get Token** — POST /auth/token ( authorization\_code , client\_credentials , or refresh\_token )

#### FHIR Resources Used

| Resource         | Access       | Purpose                                               |
|------------------|--------------|-------------------------------------------------------|
| Organization     | read         | Facilities / clinics available for booking            |
| PractitionerRole | read         | Providers linked to each organization                 |
| Schedule         | read         | Schedule envelope for a provider within a date range  |
| Slot             | read         | Free time windows within a Schedule                   |
| Patient          | read + write | Look up caller by phone, create a record if not found |
| Appointment      | write        | Create and cancel appointments                        |

Resources that are **not** required (and therefore may remain disabled on the server): RelatedPerson , standalone Practitioner , Coverage , AppointmentResponse , Location , HealthcareService , Encounter .

## 4.2 Architecture Diagrams

### Overall Sequence

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as FHIR Integration
    participant API as FHIR Server

    Note over I,API: Setup (on publish)
    A->>I: project_publish_finish
    I->>API: POST /auth/token (authorization_code OR client_credentials)
    API-->>I: access_token + refresh_token
    I->>API: GET /fhir/Organization?active=true&type=prov
    API-->>I: Bundle of Organizations (facilities)
    I->>API: GET /fhir/PractitionerRole?organization:identifier={facility_id}
    API-->>I: Bundle of PractitionerRoles (providers)
    I->>A: Tools registered, enums populated

    Note over U,API: Availability Check
    U->>A: "Any slots on Monday?"
    A->>I: get_available_slots (date, time)
    I->>API: GET /fhir/Schedule?date=ge...&date=le...&actor:identifier={provider}
    API-->>I: Bundle of Schedules
    I->>API: GET /fhir/Slot?schedule:identifier={schedule}&status=free&start=ge...&start=le...
```

```

API-->>I: Bundle of free Slots
I->>A: result_success (availability[])
A->>U: "Here are the available times..."

Note over U,API: Booking
U->>A: "Book 2:00 PM please"
A->>I: book_slot (customerInfo, bookingDateTime)
I->>API: GET /fhir/Patient?phone=...
API-->>I: Bundle (empty or existing Patient)
alt Patient not found
    I->>API: POST /fhir/Patient (name, telecom phone+email)
    API-->>I: Patient.id
end
I->>API: POST /fhir/Appointment (status=proposed, start, end, participants)
API-->>I: Appointment.id
I->>A: result_success (booking_id, contact_id)
A->>U: "Your appointment is confirmed!"

Note over U,API: Cancellation
U->>A: "Cancel my appointment"
A->>I: convo_agent_cancel_booking (booking_id)
I->>API: PATCH /fhir/Appointment/{id}
[{"op":"replace","path":"/status","value":"cancelled"}]
API-->>I: updated Appointment
I->>A: result_success (cancel_booking)
A->>U: "Your appointment has been cancelled"

```

## AvailabilityFlow

```

flowchart TD
    E["get_available_slots"] --> CHK1["CHK{Slot check<br>enabled?}"]
    CHK1 -->|"No"| SKIP["Return (disabled)"]
    CHK1 -->|"Yes"| PARSE1["Parse payload:<br>date, time"]
    PARSE1 --> ATTR["ATTR{Read attributes:<br>base_url, provider,<br>show_for_days}"]
    ATTR --> CALC1["CALC{Calculate date range:<br>date_from = requested date<br>date_to = date_from + show_for_days}"]
    CALC1 --> SCH["SCH{GET /fhir/Schedule<br>?date=ge...&date=le...<br>&actor:identifier={provider_id}}"]
    SCH --> SOK1["SOK{Schedule<br>found?}"]
    SOK1 -->|"No"| ERR1["result_error"]
    SOK1 -->|"Yes"| SLOT["SLOT{GET /fhir/Slot<br>?schedule:identifier={schedule}<br>&status=free<br>&start=ge...&start=le...}"]
    SLOT --> OK1["OK{Success?}"]
    OK1 -->|"No"| ERR2["ERR"]
    OK1 -->|"Yes"| FMT["FMT{Format slots:<br>group by date,<br>convert to HH:MM AM/PM}"]
    FMT --> OUT["OUT{result_success<br>{availability[]}}"]

```

## BookingFlow

flowchart TD

```
E["book_slot"] --> CHK{"Booking<br>enabled?"}
CHK -->|"No"| SKIP["Return (disabled)"]
CHK -->|"Yes"| PHONE["Extract phone from<br>customerInfo.phoneNumber"]
PHONE --> SEARCH["GET /fhir/Patient<br>?phone={phoneNumber}"]
SEARCH --> FOUND{"Patient<br>found?"}
FOUND -->|"Yes"| USE["Use existing<br>Patient.id"]
FOUND -->|"No"| CREATE["POST /fhir/Patient<br>{name.given, name.family,
<br>telecom: phone + email}"]
CREATE --> NEW["Extract new<br>Patient.id"]
USE --> BOOK["POST /fhir/Appointment<br>{resourceType: Appointment,<br>status:
proposed,<br>start, end (+1h),<br>participants: Practitioner (NPI),
<br>Patient/{id}}"]
NEW --> BOOK
BOOK --> OK{"Success?"}
OK -->|"No"| ERR["result_error"]
OK -->|"Yes"| OUT["result_success<br>{booking_id, contact_id}"]
```

#### CancellationFlow

flowchart TD

```
E["convo_agent_cancel_booking"] --> CHK{"Cancellation<br>enabled?"}
CHK -->|"No"| SKIP["Return (disabled)"]
CHK -->|"Yes"| VAL{"booking_id<br>present?"}
VAL -->|"No"| ERR["result_error:<br>missing booking_id"]
VAL -->|"Yes"| API["PATCH /fhir/Appointment/{id}<br>[{op:replace, path:/status,
<br>value:cancelled}]"]
API --> OK{"Response:<br>Appointment updated?"}
OK -->|"No"| ERR2["result_error"]
OK -->|"Yes"| OUT["result_success<br>{cancel_booking}"]
```

#### UtilsFlow (Token & Request Management)

flowchart TD

```
REQ["fhir_execute_request"] --> TOK{"access_token<br>available?"}
TOK -->|"No"| REFRESH["POST /auth/token"]
TOK -->|"Yes"| HTTP["Send via<br>fhir_connector"]
HTTP --> STATUS{"HTTP<br>status?"}
STATUS -->|"200/201"| CHECK{"Response has<br>OperationOutcome error?"}
CHECK -->|"No"| SUCCESS["fhir_result_success"]
CHECK -->|"Yes"| ERR["fhir_result_error"]
STATUS -->|"401"| REFRESH
REFRESH --> GRANT{"refresh_token<br>available?"}
GRANT -->|"Yes"| REF_GRANT["grant_type=refresh_token"]
GRANT -->|"No"| CC_GRANT["grant_type=client_credentials"]
REF_GRANT --> TOK_OK{"New token?"}
CC_GRANT --> TOK_OK
TOK_OK -->|"Yes"| SAVE["Save access_token +<br>refresh_token,<br>retry request
(max 3)"]
```

```
TOK_OK -->|"No"| ERR
STATUS -->|"4xx/5xx"| ERR
```

### 4.3 Where to Test

Testing can be done through the **Newo conversation interface** (voice or chat). Each flow (Availability, Booking, Cancellation) has a dedicated test skill ( `CheckAvailabilityTestSkill` , `CreateAppointmentTestSkill` , `CancelAppointmentTestSkill` ) that can be triggered directly for connectivity validation without running the full conversational flow.

### 4.4 Testing and Verification

#### To test the integration:

1. **Setup:** Publish the project and verify the OAuth token exchange succeeds (check `fhir_access_token` is populated after publish)
2. **Facilities:** After publish, verify `fhir_facility` enum is populated with your `Organization` entries
3. **Providers:** Select a facility and re-publish — verify `fhir_provider` enum shows the `PractitionerRole` entries for that facility
4. **Availability:** Ask "What slots are available on Monday?" — verify slots are returned with correct times
5. **Booking:** Complete a booking conversation — verify the `Appointment` resource appears on the FHIR server and is linked to a valid `Patient` reference
6. **Cancellation:** Request cancellation of a booked appointment — verify the `Appointment.status` changes to `cancelled` on the server

#### If no action occurs:

- Ensure `fhir_base_url` , `fhir_client_id` , and `fhir_client_secret` are set correctly
- Verify `fhir_grant_type` matches what the server actually supports; if `authorization_code` , make sure `fhir_oauth_code` is populated and still valid (authorization codes are short-lived)
- Confirm `fhir_access_token` was obtained after setup (it should not be empty)
- Ensure the relevant feature flags are enabled: `fhir_enable_booking` , `fhir_enable_slot_check` , `fhir_enable_cancellation`
- Verify `fhir_facility` and `fhir_provider` were populated during setup — both attributes are stored in the format `id|display_name` and split on `|`
- Check that the FHIR server is reachable over HTTPS from the platform and that the configured scopes include all required resources
- Confirm the server supports the exact search parameters used by the integration ( `Organization?active=true&type=prov` , `PractitionerRole?organization:identifier=...` , `Schedule?actor:identifier=...` , `Slot?schedule:identifier=...&status=free` )
- Re-publish the project if any credentials were changed

*Note: This integration performs real operations against the configured FHIR server. Appointments created or cancelled through the agent are reflected in the live EHR.*

## 5. FAQ

**Q: Which FHIR version is supported?** A: FHIR **R4**. The integration uses R4 search parameter names ( `actor:identifier` , `schedule:identifier` ) and JSON Patch for status updates. Earlier versions (DSTU2, STU3) are not supported.



**Q: Which FHIR servers have been tested?** A: The reference implementation targets **Aidbox**, but the integration only uses standard FHIR REST and SMART-on-FHIR OAuth, so any R4-compliant server should work provided it supports the search parameters listed above. The expected token endpoint is `POST {base_url}/auth/token` — if your server exposes a different path (e.g., `/oauth2/token`), the `fhir_base_url` or the skill's token URL must be adjusted.

**Q: How does facility and provider selection work?** A: On publish, the `SetupFlow` fetches all `Organization` entries (filtered by `active=true&type=prov`) and populates the `fhir_facility` enum. After you pick a facility and re-publish, it fetches the `PractitionerRole` entries linked to that organization and populates the `fhir_provider` enum. Both attributes are stored as `"{id}|{display_name}"` strings.

**Q: What patient information is required for booking?** A: First name, last name, email, and phone number are required (validated in `_createContactSkill`). If any are missing the booking flow returns an error before touching the FHIR server.

**Q: How is patient deduplication handled?** A: The `BookingFlow` searches `Patient` by the caller's phone number via `GET /fhir/Patient?phone=...`. If a match is found, the existing `Patient.id` is reused; otherwise a new `Patient` is created. If you need deduplication on other identifiers (email, MRN, SSN), the `_getContactSkill` would need to be extended.

**Q: What OAuth2 grants are supported?** A: Both `authorization_code` (interactive one-time login through `https://static.newo.ai/oauth/index.html`) and `client_credentials` (headless, system-level). The grant type is selected via the `fhir_grant_type` attribute. Token refresh uses `refresh_token` when available, with fallback to `client_credentials`.

**Q: What happens if the OAuth token expires?** A: The `UtilsFlow` detects `401` responses from any resource call. If a refresh token is available, it calls `POST /auth/token` with `grant_type=refresh_token`. Otherwise it falls back to `grant_type=client_credentials` using the stored client ID and secret. The original request is then retried (up to 3 attempts).

**Q: How many days of availability does the agent check?** A: Configurable via the `fhir_show_for_days_customer` attribute (default: 5). The `Schedule` and `Slot` queries use `date=ge{from}` and `date=le{to}` where `to = from + show_for_days`.

**Q: What is the default appointment duration?** A: 1 hour. The `BookingFlow` sets `start = bookingDateTime` and `end = start + 1h`. If you need a different default, adjust `_createAppointmentSkill.nsl`.

**Q: How is the practitioner identified in the created Appointment?** A: The integration writes the provider into `Appointment.participant[].actor` using an NPI identifier (`system = http://hl7.org/fhir/sid/us-npi`) derived from the selected `fhir_provider` attribute. If your server uses a different identifier system for providers, `_createAppointmentSkill` must be updated accordingly.

**Q: Why is the cancellation a PATCH and not a DELETE ?** A: FHIR best practice for appointment cancellation is to preserve the record and update its `status` to `cancelled`, not to physically delete the resource. This keeps audit history intact and matches the behavior expected by most EHRs. The integration uses JSON Patch (`PATCH` with `Content-Type: application/json-patch+json`) to set `status = cancelled`.

**Q: Does the integration support rescheduling?** A: Not as a single action. Reschedule is currently handled as cancel + rebook. Adding a true `PATCH /Appointment/{id}` with new `start / end` would require a new flow.

**Q: Does the integration support multiple facilities simultaneously?** A: No. A single facility and provider are selected during setup. To switch facilities, update the `fhir_facility` attribute and re-publish to refresh the provider list.

**Q: What minimum SMART scopes does the integration actually need?** A:

```
system/Patient.read system/Patient.write
system/Organization.read
system/PractitionerRole.read
system/Schedule.read
system/Slot.read
system/Appointment.write
```

These are the minimum scopes needed to cover *retrieve customer details* and *schedule appointments*. Enabling additional resources ( `Coverage` , `Location` , `RelatedPerson` , etc.) is not required for the current feature set.